

Diseño y Análisis de Algoritmos

Ayudantía 1: Correctitud de algoritmos

Ayte. Cristian Nettle

1. Subarray Sum Equals K

Considere el siguiente algoritmo para contar el número de subarreglos cuya suma es igual a:

SUBARRAY-SUM-K(A, k)

```

1  Initialize hash table  $H$  with  $H[0] = 1$ 
2  Compute prefix sums  $S[0 \dots n-1]$  where  $S[i] = \sum_{j=0}^i A[j]$ 
3   $ans = 0$ 
4  for  $i = 0$  to  $n-1$ 
5      if  $H.contains(S[i] - k)$ 
6           $ans = ans + H[S[i] - k]$ 
7       $H[S[i]] = H[S[i]] + 1$ 
8  return  $ans$ 
```

Demuestre la correctitud del algoritmo usando invariantes de loop.

Recuerde mostrar las tres propiedades: *inicialización*, *mantención* y *terminación*.

Pauta

Para demostrar la correctitud se definirá dos invariantes:

Invariante $I_1(i)$: Al inicio de la iteración i , H contiene la cuenta de prefijos de $A[0 \dots i-1]$.

Invariante $I_2(i)$: Al inicio de la iteración i , ans contiene el conteo de subarreglos (no vacíos) que sumen k para $A[0 \dots i-1]$.

Inicialización. Para $i = 0$:

- $H = \{0 : 1\}$ contiene el conteo de prefijos de $A[0 \dots -1]$ (prefijo vacío). Por lo tanto, $I_1(0)$ se cumple.
- $ans = 0$ para $A[0 \dots -1]$, el cual es vacío, por lo que efectivamente no existen subarreglos (no vacíos) con suma k . Por lo tanto, $I_2(0)$ se cumple.

Mantenimiento. Supongamos $I_1(i)$, $I_2(i)$ son verdaderos al inicio de la iteración i .

Para I_2 :

- Si $S[i] - k$ no está en H , entonces no existe un subarreglo $A[j \dots i]$, $j = \{0, \dots, i-1\}$ con suma k en $A[0 \dots i]$. Por lo que ans no cambia.
- Si $S[i] - k$ está en H , entonces existen $H[S[i] - k]$ subarreglos $A[j \dots i]$, $j = \{0, \dots, i-1\}$ con suma k en $A[0 \dots i]$. Por lo que ans aumenta por la cantidad de prefijos que contenga $H[S[i] - k]$.
- En ambos casos, ans cuenta correctamente los subarreglos en $A[0 \dots i]$ que sumen k .

Para I_1 :

- En el cuerpo del loop se ejecuta $H[S[i]] = H[S[i]] + 1$, incrementando el conteo en $H[S[i]]$ en 1. Por lo que al inicio de la siguiente iteración H contiene exactamente los conteos de prefijos de $A[0...i]$.

Por lo tanto, al inicio de la siguiente iteración, $I_1(i+1)$, $I_2(i+1)$ se mantienen.

Terminación. El for recorre exactamente los índices $i = 0, \dots, n-1$; al finalizar $i = n$. Por $I_2(n)$ *ans* almacena el conteo de subarreglos que sumen k para $A[0...n-1]$, que es precisamente el valor que se desea.

2. Two-Sum

Considere el siguiente algoritmo para resolver el problema de Two-Sum:

TWO-SUM-HASHTABLE(A, k)

```

1  Create empty hash table  $H$ 
2  for  $i = 0$  to  $n - 1$ 
3       $c = k - A[i]$ 
4      if  $H.contains(c)$ 
5          return  $\{H[c], i\}$ 
6       $H[A[i]] = i$ 
7  return NIL
```

Demuestre la correctitud del algoritmo usando un invariante de loop.

Su invariante debe cumplir las tres propiedades: *inicialización*, *mantención* y *terminación*.

Pauta

El algoritmo devuelve un par de índices, tal que : $A[i] + A[j] = k, i \neq j$, NIL cualquier otro caso.

Invariante $I(i)$: Al inicio de la iteración i , H contiene exactamente los pares $(A[j], j)$ para todo $0 \leq j < i$. Además, no existe un par de índices $p, q < i$ tal que $A[p] + A[q] = k$.

Inicialización. Para $i = 0$, Inicialmente, H es vacío y el prefijo $A[0... - 1]$ también es vacío, entonces no existe ningún par, por lo tanto $I(0)$ se cumple.

Mantenimiento. Suponga $I(i)$ es verdadero al inicio de la iteración i .

- Si $k - A[i]$ no está en H , entonces no hay un par de elementos con suma k en $A[0...i]$. Luego se inserta $A[i]$ en H . Por lo tanto, al inicio de la siguiente iteración se cumple $I(i+1)$.
- Si $k - A[i]$ está en H , se encontró un par que cumpla la condición, por lo que el for no continúa, la corrección de este retorno se justifica en **Terminación**.

Terminación. El for recorre exactamente los índices $i = 0, \dots, n-1$; al finalizar $i = n$. El algoritmo puede terminar de dos formas:

- Si existe $q \in \{0, \dots, n-1\}$, con $c = k - A[q]$ y $p = H[c]$, entonces en la iteración $i = q$ el algoritmo retorna (p, q) . El retorno es correcto dado que $k - A[q] = A[p] \Leftrightarrow A[q] + A[p] = k$.
- Si el loop finaliza sin retorno, entonces por $I(n)$ no existe un par de índices $p, q < n$ tal que $A[p] + A[q] = k$, por lo tanto la salida NIL es correcta.